ПРАКТИЧНА РОБОТА №6

Тема: Опрацювання даних власного вебпроєкту за допомогою змінних, умов, циклів, функцій, масивів, об'єктів і сучасних методів JavaScript.
Мета: Навчитися моделювати предметні дані масивом об'єктів, перевіряти значення, будувати умови й цикли, проєктувати невеликі функції, застосовувати find, filter, map, reduce, spread і destructuring, формувати підсумкові результати та перевіряти логіку на граничних випадках без залежності від DOM.

ХІД РОБОТИ

1. Створено набір із восьми демонстраційних новин у `scripts/data.js`. Записи мають однакові поля: `id`, `title`, `text`, `category`, `date`, `priority`, `isVisible` та `isPinned`. Значення ID унікальні; персональні дані не використовуються.

```
export const news = [
  {
    id: 1,
    title: "Початок нового навчального року",
    text: "Навчальний рік у коледжі розпочинається урочистою зустріччю студентів і викладачів.",
    category: "оголошення",
    date: "2026-09-01",
    priority: 5,
    isVisible: true,
    isPinned: true
  },
  { id: 2, title: "Тиждень комп'ютерних наук", text: "На студентів чекають лекції, командні завд
```

2. У `scripts/news-service.js` реалізовано нормалізацію назви й категорії, перевірку запису та створення валідної новини. Перевіряються довжина назви й тексту, допустима категорія, коректна календарна дата, пріоритет і boolean-поля.

```
export function normalizeTitle(value) {
  return typeof value === "string" ? value.trim().replace(/\s+/g, " ") : "";
}

export function normalizeCategory(value) {
  return typeof value === "string" ? value.trim().toLocaleLowerCase("uk-UA") : "";
}
export function createNews(input, id) {
  const normalized = { ...input, id, title: normalizeTitle(input?.title), category: normalizeCategory(input?.category) };
  const result = validateNews(normalized);
  if (!Number.isInteger(id) || id < 1)
    result.errors.id = "ID має бути додатним цілим числом.";

  if (Object.keys(result.errors).length)
    throw new TypeError(`Некоректна новина: ${Object.values(result.errors).join(" ")}`);

  return normalized;
}
```

Змін	Лист	№ докум.	Підпис	Дата	ПР6.121.18		
Розроб.		Прищепний В.А.			Основи програмування мовою JavaScript	Літ.	Лист 1 / Листів 6
Перевір.		Пелихівський Л.					
Реценз.						БФКПЕП П-425	
Н. Контр.							
Затверд.							

```
export function validateNews(input) {
  const errors = {};
  if (!input || typeof input !== "object" || Array.isArray(input)) {
    return { isValid: false, errors: { input: "Очікується об’єкт новини." } };
  }
  const title = normalizeTitle(input.title);

  if (title.length < 3 || title.length > 80)
    errors.title = "Назва має містити від 3 до 80 символів.";

  if (typeof input.text !== "string" || input.text.trim().length < 10 || input.text.trim().length > 1000) {
    errors.text = "Текст має містити від 10 до 1000 символів.";
  }

  if (!CATEGORIES.has(normalizeCategory(input.category)))
    errors.category = "Оберіть одну з категорій: оголошення, подія, навчання.";

  if (typeof input.date !== "string" || !/^\d{4}-\d{2}-\d{2}$/.test(input.date) || Number.isNaN(Date.parse(`${input.date}T00:00:00Z`)) ||
    errors.date = "Дата має бути коректною датою у форматі РРРР-ММ-ДД.";
  }

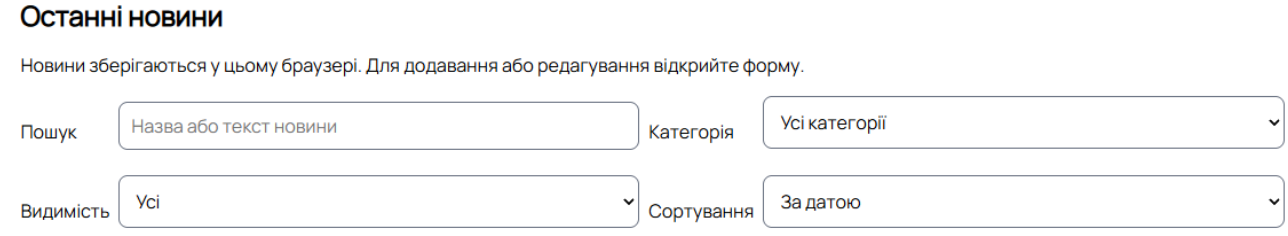
  if (!Number.isInteger(input.priority) || input.priority < 1 || input.priority > 5)
    errors.priority = "Пріоритет має бути цілим числом від 1 до 5.";

  if (typeof input.isVisible !== "boolean")
    errors.isVisible = "Видимість має бути логічним значенням.";

  if (typeof input.isPinned !== "boolean")
    errors.isPinned = "Закріплення має бути логічним значенням.";
  return { isValid: Object.keys(errors).length === 0, errors };
}
```

3. Реалізовано пошук новини за ID, фільтр за категорією, текстовий пошук без урахування регістру, комбінований фільтр за категорією та видимістю, а також перевірку наявності видимої закріпленої новини.

```
export const findNewsById = (items, id) => items.find((item) => item.id === id);
export const filterByCategory = (items, category) => items.filter((item) => item.category === normalizeCategory(category));
export const searchNews = (items, query) => {
  const normalizedQuery = typeof query === "string" ? query.trim().toLocaleLowerCase("uk-UA") : "";
  return items.filter((item) => `${item.title} ${item.text}`.toLocaleLowerCase("uk-UA").includes(normalizedQuery));
};
export const filterNews = (items, { category, isVisible } = {}) => items.filter((item) =>
  (category === undefined || item.category === normalizeCategory(category)) &&
  (isVisible === undefined || item.isVisible === isVisible));
```



4. Реалізовано сортування за назвою та датою без мутації початкового масиву.

```
export const sortByTitle = (items) => [...items].sort((a, b) => a.title.localeCompare(b.title, "uk"));
export const sortByDate = (items) => [...items].sort((a, b) => a.date.localeCompare(b.date));
```

5. Додано операції додавання, оновлення та видалення. Дубльований ID і некоректні записи відхиляються; під час оновлення ID запису зберігається. Поведінку для відсутнього ID описано в `README.md`.

```
export function updateNews(items, itemId, changes) {
  const index = items.findIndex((item) => item.id === itemId);

  if (index === -1)
    return items;

  const updated = { ...items[index], ...changes, id: items[index].id };
  updated.title = normalizeTitle(updated.title);
  updated.category = normalizeCategory(updated.category);
  const result = validateNews(updated);

  if (!result.isValid)
    throw new TypeError("Не можна зберегти некоректну новину.");

  return items.map((item, currentIndex) => currentIndex === index ? updated : item);
}

export const removeNews = (items, itemId) => items.filter((item) => item.id !== itemId);
```

6. Створено 19 автоматизованих self-checks у `scripts/news-service.test.js`. Перевірено звичайні й граничні випадки: набір ідентифікаторів, нормалізацію, невалідні поля, пошук, фільтри, незмінність сортування та CRUD

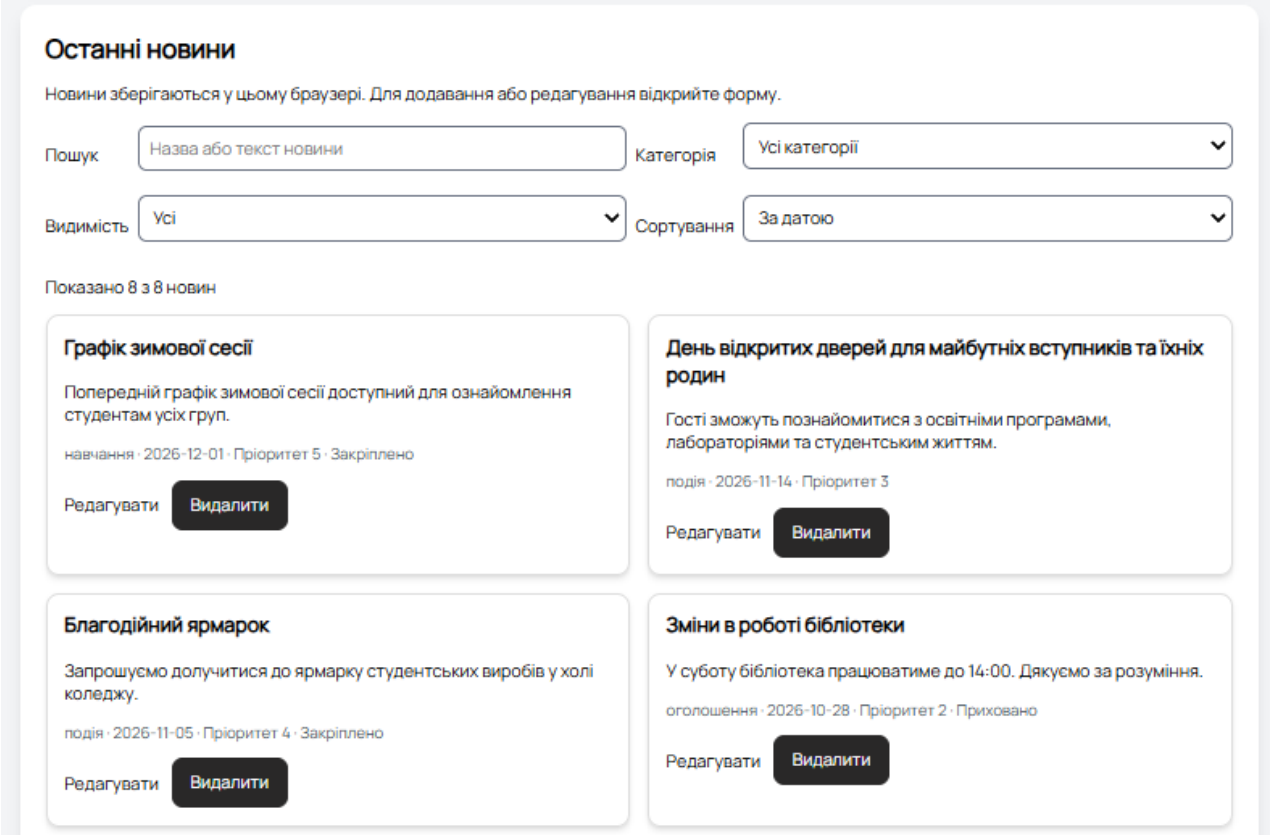
```
const checks = [];
function check(name, fn) {
  try { fn(); checks.push({ name, passed: true }); }
  catch (error) { checks.push({ name, passed: false, error }); }
}

const draft = { title: "  Тестова   новина ", text: "Достатньо довгий текст новини для перевірки.", category: " Подія ", date: "2026-1
check("набір містить щонайменше 8 записів з унікальними ID", () => { assert.ok(news.length >= 8); assert.equal(new Set(news.map((item)
check("нормалізація назви прибирає зайві пробіли", () => assert.equal(normalizeTitle(draft.title), "Тестова новина"));
check("нормалізація категорії враховує регістр", () => assert.equal(normalizeCategory(draft.category), "подія"));
check("валідний запис проходить перевірку", () => assert.equal(validateNews(createNews(draft, 100)).isValid, true));
check("коротка назва відхиляється", () => assert.ok(validateNews({ ...draft, title: "x" }).errors.title));
check("невідома категорія відхиляється", () => assert.ok(validateNews({ ...draft, category: "інше" }).errors.category));
check("неможлива календарна дата відхиляється", () => assert.ok(validateNews({ ...draft, date: "2026-02-31" }).errors.date));
check("пріоритет поза діапазоном відхиляється", () => assert.ok(validateNews({ ...draft, priority: 6 }).errors.priority));
check("пошук за ID повертає запис або undefined", () => { assert.equal(findNewsById(news, 1), news[0]); assert.equal(findNewsById(news
```

```
Підсумок: 19/19 перевірок пройдено.

Демонстраційний звіт по новинах
Усього: 8; видимих: 7; середній пріоритет: 3.88.
За категоріями: оголошення — 2, подія — 3, навчання — 3.
Є видима закріплена новина: так.
Приклад картки: {
  id: 1,
  title: 'Початок нового навчального року',
  category: 'оголошення',
  date: '2026-09-01',
  priority: 5,
  isPinned: true
}
```

7. На `index.html` виводиться список новин із пошуком, фільтрами, сортуванням, редагуванням і видаленням. `news-form.html` використовується для створення та редагування новин.



8. Додано `scripts/news-storage.js` для завантаження початкових даних і збереження змін у `localStorage` браузера. Додано стилі карток і панелі керування в `styles/main.css`.

```
const STORAGE_KEY = "bfkpep-news-v1";

export function loadNews() {
  try {
    const saved = localStorage.getItem(STORAGE_KEY);

    if (saved === null)
      return initialNews.map((item) => ({ ...item }));

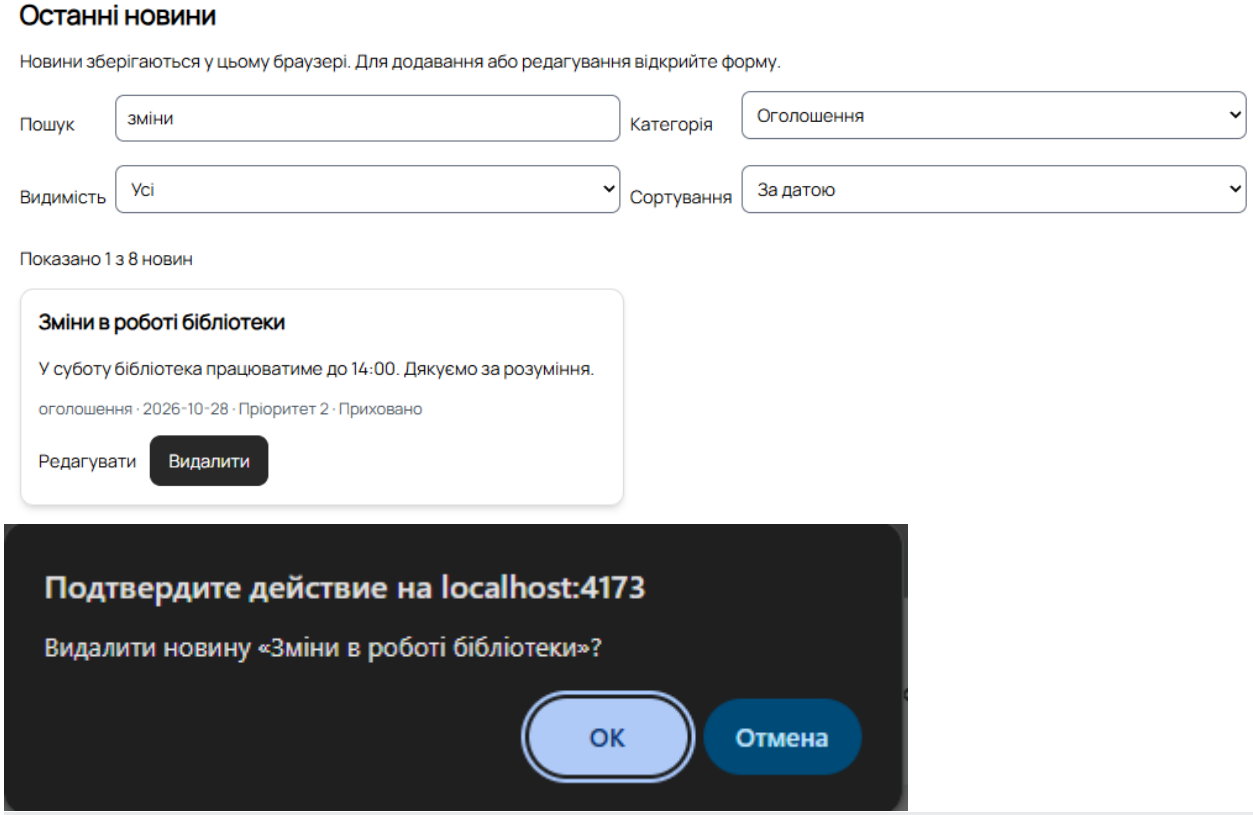
    const parsed = JSON.parse(saved);

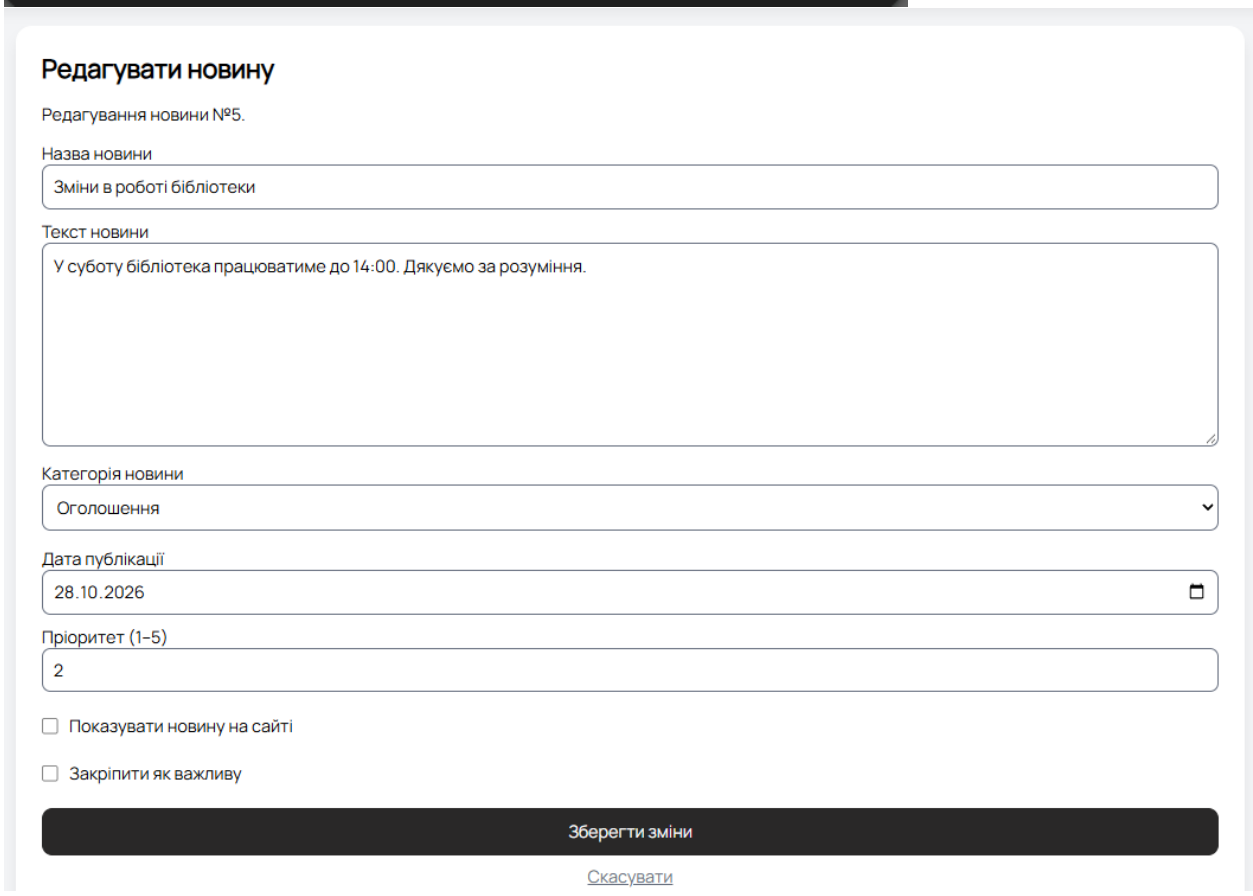
    if (!Array.isArray(parsed))
      return initialNews.map((item) => ({ ...item }));

    const ids = parsed.map((item) => item?.id);
    if (new Set(ids).size !== parsed.length ||
        !parsed.every((item) => Number.isInteger(item?.id) && validateNews(item).isValid)) {
      return initialNews.map((item) => ({ ...item }));
    }
    return parsed;
  } catch {
    return initialNews.map((item) => ({ ...item }));
  }
}

export function saveNews(items) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(items));
}
```

9. Додано `scripts/dev-server.js` і команду `npm run serve`. Локальний сервер потрібен, оскільки браузер блокує ES-модулі при відкритті сторінки через `file://`.
10. Скріншоти результатів роботи програми:





Контрольні питання

1. Основну сутність зручно представляти об'єктом, оскільки він об'єднує всі властивості одного елемента.
2. Нормалізація приводить дані до потрібного формату, а валідація перевіряє їхню правильність.
3. find використовують для пошуку першого елемента, що відповідає умові, а filter — для отримання всіх таких елементів.
4. map створює новий масив із результатів перетворення елементів, а forEach виконує дію для кожного елемента.
5. У reduce використав початкове значення, яке відповідає типу результату, щоб коректно обробляти масив, зокрема порожній.
6. Функція перевіряє порожній масив і повертає коректний результат без помилок.
7. sort змінює початковий масив, тому для збереження оригінальних даних потрібно спочатку створити його копію.
8. Spread (...) дозволив створювати копію об'єкта та змінювати потрібні властивості без мутації початкового об'єкта.
9. Self-checks допомогли виявити порожні масиви, відсутні елементи, некоректні дані та граничні значення.
10. У DOM-інтерфейсі повторно використаю функції пошуку, фільтрації, сортування, обчислення та оновлення даних.